ratvalue

User Manual

Damodaran Valuation Framework in C++23

sheep-farm

June 2026 `v1.0`

Contents

Abstract 4

Quick Start 5

1 Introduction 8
- 1.1 Motivation . . . 8
- 1.2 Scope . . . 8
- 1.3 License . . . 8
- 1.4 Dependencies and Build . . . 8

2 Infrastructure: ratmoney and Internal Types 10
- 2.1 ratmoney: Exact Monetary Arithmetic . . . 10
- 2.2 types.h: Errors and Projection . . . 11
- 2.3 detail.h: Internal Rational Arithmetic . . . 11

3 Earnings Normalization 12
- 3.1 Theory . . . 12
- 3.2 API . . . 12
- 3.3 Remarks . . . 13

4 Cost of Capital 14
- 4.1 CAPM with Country Risk Premium . . . 14
  - 4.1.1 Model . . . 14
  - 4.1.2 API . . . 14
- 4.2 Beta: Hamada and Bottom-Up . . . 15
  - 4.2.1 Hamada Equation . . . 15
  - 4.2.2 Bottom-Up Beta . . . 15
  - 4.2.3 API . . . 15
- 4.3 Synthetic Cost of Debt . . . 16
  - 4.3.1 Methodology . . . 16
  - 4.3.2 Summary Table (Large Cap) . . . 16
  - 4.3.3 API . . . 17
- 4.4 WACC . . . 17
  - 4.4.1 Formula . . . 17
  - 4.4.2 API . . . 17

5 Free Cash Flow 18
- 5.1 FCFF — Free Cash Flow to Firm . . . 18
  - 5.1.1 Formula . . . 18

5.1.2 API . . . 18
5.2 FCFE — Free Cash Flow to Equity . . . 18
5.2.1 Formula . . . 18
5.2.2 Conceptual Difference: FCFF vs. FCFE . . . 19
5.2.3 API . . . 19
5.3 Fundamental Growth . . . 20
5.3.1 Theory . . . 20
5.3.2 API . . . 20

6 Discounted Cash Flow **21**
6.1 Standard Model: FCFF and WACC . . . 21
6.1.1 Two-Stage Structure . . . 21
6.1.2 Implementation: double for Discounting, Rational for TV . . . 21
6.1.3 API . . . 21
6.2 DDM — Dividend Discount Model . . . 22
6.3 H-Model (Fuller & Hsia, 1984) . . . 22

7 Terminal Value **24**
7.1 Consistency with ROIC . . . 24
7.1.1 The Problem with the Standard Gordon Model . . . 24
7.1.2 Consistent Terminal Value (Damodaran ch. 12) . . . 24
7.1.3 Consistency Audit . . . 24
7.2 TV by Multiple . . . 25

8 APV — Adjusted Present Value **26**
8.1 Motivation . . . 26
8.2 Modigliani-Miller (1963): Permanent Debt . . . 26
8.3 Miles-Ezzell (1980): Rebalanced Debt . . . 26

9 EVA and Excess Returns **28**
9.1 Theoretical Background . . . 28
9.2 Economic Interpretation . . . 28
9.3 API . . . 28

10 Justified Multiples **30**
10.1 Background . . . 30
10.2 API . . . 30

11 Optimal Capital Structure **32**
11.1 Algorithm . . . 32
11.2 API . . . 32

12 Specialized Models **34**
12.1 Scenario-Based Valuation: Startups and Transitioning Firms . . . 34
12.1.1 Motivation . . . 34
12.1.2 API . . . 34
12.2 Equity as an Option: Merton Model . . . 35
12.2.1 Background . . . 35
12.2.2 API . . . 35

12.3 FCFE for Financial Institutions . . . 36
12.3.1 Why Banks Are Different . . . 36
12.3.2 Model . . . 36
12.3.3 API . . . 36

13 Full Example: Petrobras and Itaú Unibanco **37**
13.1 Data . . . 37
13.2 Cost of Capital Parameters . . . 37
13.3 Growth and FCFF . . . 38
13.4 Summary of Results . . . 38
13.5 Itaú Unibanco (ITUB4) . . . 38

14 Data Flow Between Modules **40**
14.1 Overview . . . 40
14.2 Cross-Model Consistency . . . 40

15 Python Bindings **41**
15.1 Overview . . . 41
15.2 Installation . . . 41
15.3 Unit Conventions . . . 42
15.4 API Reference . . . 42
15.4.1 Cost of Capital . . . 42
15.4.2 Data and Cash Flows . . . 43
15.4.3 Growth . . . 43
15.4.4 Valuation Models . . . 43
15.5 Complete Example: Petrobras . . . 45

A Rating and Spread Tables (Damodaran 2023) **47**

B References and Further Reading **49**

Abstract

ratvalue is a C++23 library implementing Aswath Damodaran's complete valuation framework, from earnings normalization to specialized models for startups, distressed firms, and financial institutions. The project is built on top of `ratmoney`, which provides exact rational monetary arithmetic (`int64_t` + GCD over `__int128`).

This manual covers:

- Theoretical foundations of each valuation model;
- Complete C++23 API specification;
- Verifiable numerical examples;
- Real-world case study with Petrobras (PETR4) and Itaú Unibanco (ITUB4).

Prerequisites: familiarity with C++17+, basic linear algebra, and corporate finance at the undergraduate level.

Quick Start

This chapter gets you from zero to a working DCF in five minutes, in both C++ and Python. Read the remaining chapters for theory and full API details.

Step 1 — Prerequisites

- **ratmoney** installed in `/usr/local` (https://github.com/sheep-farm/ratmoney)
- GCC $\geq$ 13 or Clang $\geq$ 17, CMake $\geq$ 3.20, GTest
- *Python bindings only:* Python $\geq$ 3.9, nanobind $\geq$ 2.0, scikit-build-core $\geq$ 0.9

Step 2 — Build

Listing 1: C++ library, tests, and demo binary

```
git clone https://github.com/sheep-farm/ratvalue
cd ratvalue
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
ctest --test-dir build --output-on-failure    # 82 tests, all green
./build/main_bin                              # Petrobras + Itau full
    example
```

Listing 2: Python bindings — virtual environment

```
python3 -m venv venv && source venv/bin/activate
pip install nanobind scikit-build-core
pip install --no-build-isolation .
```

Listing 3: Python bindings — system-wide (Arch Linux)

```
pip install --break-system-packages nanobind scikit-build-core
pip install --break-system-packages --no-build-isolation .
```

Step 3 — Minimal DCF in C++

Listing 4: Minimal two-stage DCF — C++

```
#include "fcff.h"
#include "capm.h"
#include "wacc.h"
```

```cpp
#include "dcf.h"
#include "detail.h"

using namespace ratvalue;
using ratmoney::Currency, ratmoney::Rational, ratmoney::
    CurrencyDescription;

int main() {
    CurrencyDescription BRL{"BRL", "R$", 2};
    Rational ONE{1, 1};
    auto B = [&](int64_t b) {          // b = billions
        return Currency{b * 100'000'000'000LL, ONE, BRL};
    };

    // Cost of capital
    Rational ke = *cost_of_equity({
        .risk_free_rate      = {1, 20},   // 5%
        .beta                = {13, 10},  // 1.30
        .equity_risk_premium = {1, 20},   // 5%
    });
    Rational wacc = *compute_wacc({
        .equity_weight  = {2, 3},
        .debt_weight    = {1, 3},
        .cost_of_equity = ke,
        .cost_of_debt   = {6, 100},  // 6%
        .tax_rate       = {34, 100},
    });

    // FCFF base and projection: 3% for 5 years
    Currency fcff0 = *compute_fcff({
        .ebit                     = B(126),
        .tax_rate                 = {34, 100},
        .depreciation_amortization = B(45),
        .capex                    = B(70),
        .delta_nwc                = B(5),
    });
    auto fcffs = *project_fcff({
        .base_fcff = fcff0,
        .stages    = {{{3, 100}, 5}},
    });

    // DCF
    auto r = *compute_dcf({
        .projected_fcff     = fcffs,
        .wacc               = wacc,
        .terminal_growth    = {3, 200},   // 1.5%
        .net_debt           = B(250),
        .shares_outstanding = 13'000'000'000LL,
    });

    double price = detail::to_double(r.equity_value_per_share);
    // -> ~39.88
}
```

Step 4 — Minimal DCF in Python

Listing 5: Equivalent DCF in Python

```python
import ratvalue as rv

fcffs = rv.project_fcff(
    rv.compute_fcff(ebit_b=126, tax_rate=0.34, da_b=45,
                    capex_b=70, delta_nwc_b=5),
    stages=[(0.03, 5)]
)
ke   = rv.cost_of_equity(rf=0.05, beta=1.30, erp=0.05)
wacc = rv.compute_wacc(2/3, ke, 1/3, kd=0.06, tax_rate=0.34)

result = rv.compute_dcf(fcffs, wacc=wacc, terminal_growth=0.015,
                        net_debt_billions=250, shares=13_000_000_000
                           )
print(result.price_per_share)   # ~39.89
```

Step 5 — Where to go next

Chapter 4.1 Extended CAPM with country risk premium — essential for emerging-market valuations.

Chapter 6 Anchoring growth in ROIC and reinvestment rate — avoids the most common DCF error.

Chapter 8 Consistent terminal value — links g, ROIC, and FCFF at the stable stage.

Chapter 15 Python bindings — full API reference and complete Petrobras example in Python.

Chapter 1

Introduction

1.1 Motivation

Valuation models are the analytical core of every capital allocation decision. Damodaran's framework, developed over three decades in *Investment Valuation* (3rd ed., 2012) and *The Dark Side of Valuation* (3rd ed., 2018), is the most rigorous and comprehensive reference available.

Existing implementations focus on Python or spreadsheets, which have two significant shortcomings for serious quantitative analysis:

1. **Numerical precision:** `float64` cannot represent decimal fractions exactly. Errors accumulate across chained calculations.
2. **Performance:** Python has the GIL and boxing overhead; spreadsheets are neither version-controlled nor auditable as code.

ratvalue addresses both: exact monetary arithmetic via `ratmoney`, and compiled C++ performance with `-O2`.

1.2 Scope

Table 1.1 lists all implemented modules.

1.3 License

ratvalue is distributed under the **MIT License**.

Use, modification, distribution, and commercial use are freely permitted with no restrictions beyond attribution. The full license text is available in the `LICENSE` file at the repository root.

1.4 Dependencies and Build

Dependencies:

- `ratmoney` ≥ 0.1 (installed in `/usr/local`)

Table 1.1: ratvalue modules

File	Category	Contents
`normalize`	Data	EBIT normalization (historical / margin)
`capm`	Cost of capital	CAPM with CRP (emerging markets)
`beta`	Cost of capital	Hamada equation; bottom-up beta
`kd`	Cost of capital	Synthetic K_d via ICR $\rightarrow$ rating
`wacc`	Cost of capital	WACC
`capital_structure`	Structure	Optimal capital structure
`fcff`	Cash flows	FCFF; multi-stage projection
`fcfe`	Cash flows	FCFE; equity DCF
`growth`	Growth	ROIC, reinvestment rate, fundamental growth
`dcf`	Valuation	DCF (FCFF $\rightarrow$ WACC $\rightarrow$ EV)
`ddm`	Valuation	Multi-stage DDM; H-Model
`terminal`	Valuation	Consistent TV; TV by multiple
`apv`	Valuation	APV (MM and Miles-Ezzell)
`excess_returns`	Analysis	EVA / Excess Returns
`relative`	Analysis	Justified P/E, P/BV, EV/EBITDA
`startup`	Specialized	Scenario-based valuation
`distress`	Specialized	Equity as an option (Merton)
`bank`	Specialized	FCFE for banks (regulatory capital)

- GCC $\geq$ 13 or Clang $\geq$ 17 (C++23)
- CMake $\geq$ 3.20
- GTest (for the test suite)

Listing 1.1: Build and test

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j$(nproc)
ctest --test-dir build --output-on-failure
```

Chapter 2

Infrastructure: ratmoney and Internal Types

2.1 ratmoney: Exact Monetary Arithmetic

Every monetary unit in **ratvalue** is represented by the type `ratmoney::Currency`, which stores:

- `units`: count in the smallest denomination (*cents*, for BRL and USD with `precision=2`);
- `rate`: the value of 1 unit of this currency in units of an implicit base currency (type `Rational`);
- `description`: name, symbol, and decimal precision.

The type `ratmoney::Rational` represents a rational number p/q in canonical form: $q > 0$, $\gcd(|p|, q) = 1$. All arithmetic uses `__int128` as an intermediate, eliminating silent overflow.

Listing 2.1: Creating currency values in ratvalue

```
ratmoney::CurrencyDescription BRL{"BRL", "R$", 2};
ratmoney::Rational UNIT_RATE{1, 1};

// BRL 1 billion = 1e11 cents
Currency B(int64_t billions) {
    return Currency{billions * 100'000'000'000LL, UNIT_RATE, BRL};
}

Currency ebit = B(145);   // BRL 145 billion
```

Warning

Currency identity is determined by `CurrencyDescription`. Two `Currency` objects can only be added or compared if they share the *same description*. The `rate` field is for *currency conversion*, not for denomination identification. ratvalue validates currency consistency in all relevant functions and returns `ValuationError::CurrencyMismatch` on violation.

2.2 types.h: Errors and Projection

Listing 2.2: Core types in ratvalue

```
enum class ValuationError {
    InvalidInput,      // invalid input (negative, zero where
        prohibited...)
    WACCLEGrowth,      // WACC <= g: Gordon denominator would be <= 0
    Overflow,          // arithmetic overflow in Rational
    MoneyError,        // CurrencyError from ratmoney
    CurrencyMismatch, // monetary inputs in different denominations
};

struct ProjectionStage {
    ratmoney::Rational growth_rate;
    int                periods;    // years in this stage
};
```

All ratvalue functions return `std::expected<T, ValuationError>`, forcing explicit error handling at compile time.

2.3 detail.h: Internal Rational Arithmetic

The internal header `detail.h` exposes helpers implementing $+$, $-$, $\times$, $\div$ over `Rational` with `__int128` intermediates:

```
namespace ratvalue::detail {
    expected<Rational, ValuationError> r_add(Rational a, Rational b);
    expected<Rational, ValuationError> r_sub(Rational a, Rational b);
    expected<Rational, ValuationError> r_mul(Rational a, Rational b);
    expected<Rational, ValuationError> r_div(Rational a, Rational b);
    expected<Rational, ValuationError> one_plus(Rational r); // 1 + r
    expected<Rational, ValuationError> make_rational(__int128 n,
        __int128 d);

    bool   same_currency(const Currency& a, const Currency& b);
    double to_double(const Currency& c);   // major units (e.g.,
        reais)
    expected<Currency, ValuationError>
        make_currency_from_double(double v, Rational rate,
                                  const CurrencyDescription& desc);
}
```

Warning

`detail.h` is an *internal* header and is not installed. Client code must not include it directly.

Chapter 3

Earnings Normalization

3.1 Theory

Cyclical firms (oil, mining, steel, agriculture) exhibit volatile EBIT that distorts any DCF based on the most recent year. Damodaran (ch. 8) recommends normalizing before estimating value, using one of two approaches:

Historical average Computes the arithmetic mean of EBIT over the last N years ($N \geq 2$), capturing the "mid-cycle EBIT":

$$EBIT_{\text{norm}} = \frac{1}{N} \sum_{i=1}^{N} EBIT_i$$

Normalized margin Applies a historical (or sector-level) operating margin to current revenues. Useful when revenues are stable but margins fluctuate:

$$EBIT_{\text{norm}} = \text{Current Revenue} \times \text{Normalized Margin}$$

3.2 API

```
enum class NormalizationMethod {
    HistoricalAverage,  // method 1
    NormalizedMargin,   // method 2
};

struct NormalizationInputs {
    NormalizationMethod              method;
    std::vector<ratmoney::Currency> historical_ebit;   // >= 2 for
        Average
    ratmoney::Currency               current_revenue;    // for Margin
    ratmoney::Rational               normalized_margin;  // for Margin
};

expected<Currency, ValuationError>
normalize_ebit(const NormalizationInputs& inputs);
```

Example: Petrobras — historical cycle average

```
auto result = normalize_ebit(NormalizationInputs{
    .method = NormalizationMethod::HistoricalAverage,
    .historical_ebit = {B(80), B(95), B(145), B(165), B(145)},
    // EBIT 2019-2023 in BRL billions
    .current_revenue = B(500),  // not used in this method
    .normalized_margin = {0, 1},
});
// result->units() / 1e11 = BRL 126B (average)
```

3.3 Remarks

The arithmetic uses accumulated `Currency::add` followed by `Currency::scale` with factor $\{1, N\}$. Rounding is *HalfEven* (banker's rounding), so the exact sum is preserved and the rounding error is at most 1 cent.

Chapter 4

Cost of Capital

4.1 CAPM with Country Risk Premium

4.1.1 Model

For emerging markets, Damodaran (*Investment Valuation*, ch. 7) extends the CAPM by adding a country risk premium weighted by the firm's exposure λ:

$$K_e = R_f + \beta \cdot \text{ERP}_{\text{mature}} + \lambda \cdot \text{CRP} \tag{4.1}$$

Where:

- R_f: global risk-free rate (US 10Y T-Bond, stripped of any differential inflation component if necessary);
- $\text{ERP}_{\text{mature}}$: equity risk premium of the reference market (typically the US; Damodaran publishes annual estimates);
- CRP: country risk premium, computed as the sovereign spread $\times$ $(\sigma_{\text{equity}}/\sigma_{\text{bond}})$;
- λ: firm's exposure to country risk. $\lambda = 1$ for firms with entirely domestic revenues; $\lambda < 1$ for exporters or multinationals.

Definition: Country Risk Premium (CRP)

$$\text{CRP} = \text{Sovereign spread} \times \frac{\sigma_{\text{equity, emerging market}}}{\sigma_{\text{sovereign bond}}}$$

For Brazil in 2024: EMBI+ spread $\approx 2\%$; volatility ratio ≈ 1.5; CRP $\approx 3\%$.

4.1.2 API

```
struct CAPMInputs {
    Rational risk_free_rate;
    Rational beta;
    Rational equity_risk_premium;              // ERP of mature
        market
    Rational country_risk_premium{0, 1};       // CRP (default: 0)
    Rational lambda{1, 1};                     // country exposure (
        default: 1)
```

```
};

expected<Rational, ValuationError>
cost_of_equity(const CAPMInputs& inputs);
```

All computation uses exact rational arithmetic. For $R_f = 5\% = \frac{1}{20}$, $\beta = 1.30 = \frac{13}{10}$, ERP $= 7\% = \frac{7}{100}$, CRP $= 3\% = \frac{3}{100}$, $\lambda = 1$:

$$K_e = \frac{1}{20} + \frac{13}{10} \cdot \frac{7}{100} + 1 \cdot \frac{3}{100} = \frac{50}{1000} + \frac{91}{1000} + \frac{30}{1000} = \frac{171}{1000} = 17.1\%$$

4.2 Beta: Hamada and Bottom-Up

4.2.1 Hamada Equation

Financial leverage amplifies systematic risk:

$$\beta_L = \beta_U \cdot \left[1 + (1 - t) \cdot \frac{D}{E}\right] \tag{4.2}$$

Inversely:

$$\beta_U = \frac{\beta_L}{1 + (1 - t) \cdot D/E} \tag{4.3}$$

4.2.2 Bottom-Up Beta

Historical beta (return regression) is noisy and reflects the past capital structure. Damodaran recommends the *bottom-up beta* approach:

1. Collect β_L from n comparable firms in the same sector;
2. Unlever each comparable via eq. (4.3);
3. Compute the average: $\bar{\beta}_U = n^{-1} \sum_i \beta_{U,i}$;
4. Re-lever to the target capital structure via eq. (4.2).

4.2.3 API

```
// Lever beta
expected<Rational, ValuationError>
lever_beta(Rational unlevered_beta, Rational debt_to_equity, Rational
    tax_rate);

// Unlever beta
expected<Rational, ValuationError>
unlever_beta(Rational levered_beta, Rational debt_to_equity, Rational
    tax_rate);

struct BottomUpBetaInputs {
    vector<Rational> comparable_levered_betas;
    vector<Rational> comparable_debt_to_equity;
    Rational         comparable_tax_rate;
    Rational         target_debt_to_equity;
    Rational         target_tax_rate;
```

```
};

expected<Rational, ValuationError>
bottom_up_beta(const BottomUpBetaInputs& inputs);
```

Warning

The average of β_U inside `bottom_up_beta` is computed in `double`, as market betas are statistical estimates without exact precision. The final re-levered result is returned as `Rational` with 4 decimal places.

4.3 Synthetic Cost of Debt

4.3.1 Methodology

Firms without traded bonds have no observable K_d. Damodaran proposes estimating it from the *interest coverage ratio* (ICR):

$$\text{ICR} = \frac{\text{EBIT}}{\text{Interest Expense}} \quad \longrightarrow \quad \text{synthetic rating} \quad \longrightarrow \quad K_d = R_f + \text{spread}(\text{rating})$$

The ICR $\rightarrow$ rating table is published annually at https://pages.stern.nyu.edu/~adamodar/. ratvalue implements the 2023 tables for *large cap* and *small cap* (tighter thresholds for smaller firms).

4.3.2 Summary Table (Large Cap)

Table 4.1: ICR $\rightarrow$ Rating $\rightarrow$ Spread (large cap, Damodaran 2023)

Min. ICR	Rating	Spread (bps)
> 8.50	Aaa/AAA	63
6.50	Aa2/AA	78
5.50	A1/A+	98
4.25	A2/A	108
3.00	A3/A–	122
2.50	Baa2/BBB	156
2.25	Ba1/BB+	200
2.00	Ba2/BB	240
1.75	B1/B+	350
1.50	B2/B	375
1.25	B3/B–	500
0.80	Caa/CCC	600
0.65	Ca2/CC	700
0.20	C2/C	800
< 0.20	D2/D	1200

4.3.3 API

```
enum class CreditRating : uint8_t {
    AAA, AA, A_plus, A, A_minus, BBB, BB_plus, BB,
    B_plus, B, B_minus, CCC, CC, C, D
};

struct SyntheticKdInputs {
    Currency ebit;
    Currency interest_expense;
    Rational risk_free_rate;
    bool     large_firm{true};
};

struct SyntheticKdResult {
    Rational     cost_of_debt;      // Kd = Rf + spread (exact
       Rational)
    Rational     default_spread;
    CreditRating rating;
    double       interest_coverage;
};

expected<SyntheticKdResult, ValuationError>
synthetic_cost_of_debt(const SyntheticKdInputs& inputs);

// direct version (for internal optimizer)
expected<SyntheticKdResult, ValuationError>
synthetic_cost_of_debt_from_icr(double icr, Rational rf, bool
   large_firm);
```

4.4 WACC

4.4.1 Formula

$$\text{WACC} = \frac{E}{V} K_e + \frac{D}{V} K_d (1 - t) \tag{4.4}$$

Where $V = E + D$.

4.4.2 API

```
struct WACCInputs {
    Rational equity_weight;   // E/V
    Rational debt_weight;     // D/V
    Rational cost_of_equity;  // Ke
    Rational cost_of_debt;    // Kd
    Rational tax_rate;        // t
};

expected<Rational, ValuationError>
compute_wacc(const WACCInputs& inputs);
```

All arithmetic is exact rational. For the Petrobras example parameters: $E_w = 2/3$, $K_e = 4629/40000$, $D_w = 1/3$, $K_d = 578/10000$, $t = 17/50$:

$$\text{WACC} = \frac{2}{3} \cdot \frac{4629}{40000} + \frac{1}{3} \cdot \frac{578}{10000} \cdot \frac{33}{50} = \frac{44933}{500000} \approx 8.99\%$$

Chapter 5

Free Cash Flow

5.1 FCFF — Free Cash Flow to Firm

5.1.1 Formula

$$\text{FCFF} = \text{EBIT}(1-t) + D\&A - \text{CapEx} - \Delta NWC \tag{5.1}$$

Where ΔNWC is the change in non-cash working capital.

5.1.2 API

```
struct FCFFInputs {
    Currency ebit;
    Rational tax_rate;
    Currency depreciation_amortization;
    Currency capex;
    Currency delta_nwc;
};

expected<Currency, ValuationError>
compute_fcff(const FCFFInputs& inputs);

struct FCFFProjection {
    Currency                    base_fcff;
    std::vector<ProjectionStage> stages;
};

// Returns [FCFF_1, FCFF_2, ..., FCFF_N] (base not included)
expected<std::vector<Currency>, ValuationError>
project_fcff(const FCFFProjection& proj);
```

The projection is exact: each period applies `Currency::scale` with the factor $(1+g)$ represented as `Rational`.

5.2 FCFE — Free Cash Flow to Equity

5.2.1 Formula

$$\text{FCFE} = NI - (1-\delta)(\text{CapEx} - D\&A + \Delta NWC) + \Delta D \tag{5.2}$$

Where $\delta = D/(D+E)$ is the debt ratio. The interpretation: equity holders finance only the $(1-\delta)$ share of net reinvestment; the rest is covered by new borrowing.

5.2.2 Conceptual Difference: FCFF vs. FCFE

Table 5.1: FCFF vs. FCFE comparison

Dimension	FCFF	FCFE
Base	EBIT (pre-interest)	Net Income (post-interest)
Discount rate	WACC	K_e
Output	Enterprise Value	Equity Value (direct)
Leverage	Captured in WACC	Captured in FCFE

Warning

FCFF and FCFE yield the same equity value when inputs are consistent (same implicit capital structure). Discrepancies signal inconsistent assumptions, not code errors.

5.2.3 API

```
struct FCFEInputs {
    Currency net_income;
    Rational debt_ratio;                    // delta = D/(D+E)
    Currency capex;
    Currency depreciation_amortization;
    Currency delta_nwc;
    Currency net_new_debt;                 // may be negative
};

expected<Currency, ValuationError>
compute_fcfe(const FCFEInputs& inputs);

struct FCFEDCFInputs {
    std::vector<Currency> projected_fcfe;
    Rational              cost_of_equity;
    Rational              terminal_growth;
    int64_t               shares_outstanding;
};

struct FCFEDCFResult {
    Currency equity_value;
    Currency equity_value_per_share;
    Currency pv_fcfe;
    Currency terminal_value;
    Currency pv_terminal_value;
};

expected<FCFEDCFResult, ValuationError>
compute_fcfe_dcf(const FCFEDCFInputs& inputs);
```

5.3 Fundamental Growth

5.3.1 Theory

Damodaran insists that growth assumptions must be *anchored* in fundamentals. "Free" growth (without reinvestment) is a common error that artificially inflates value.

$$g_{\text{firm}} = \text{ROIC} \times \text{RR} \tag{5.3}$$

$$g_{\text{equity}} = \text{ROE} \times (1 - \text{payout}) \tag{5.4}$$

Where:

$$\text{ROIC} = \frac{\text{NOPAT}}{\text{IC}} \qquad \text{RR} = \frac{\text{CapEx} - D\&A + \Delta NWC}{\text{NOPAT}}$$

5.3.2 API

```
// ROIC = NOPAT / Invested Capital
struct ROICInputs { Currency nopat; Currency invested_capital; };
expected<Rational, ValuationError> compute_roic(const ROICInputs&);

// RR = (CapEx - DA + dNWC) / NOPAT
struct ReinvestmentRateInputs {
    Currency capex, depreciation_amortization, delta_nwc, nopat;
};
expected<Rational, ValuationError>
compute_reinvestment_rate(const ReinvestmentRateInputs&);

// g = ROIC x RR
expected<Rational, ValuationError>
fundamental_growth_firm(Rational roic, Rational reinvestment_rate);

// g = ROE x retention
expected<Rational, ValuationError>
fundamental_growth_equity(Rational roe, Rational retention_ratio);
```

Chapter 6

Discounted Cash Flow

6.1 Standard Model: FCFF and WACC

6.1.1 Two-Stage Structure

Damodaran's standard DCF divides the horizon into two stages:

1. **Explicit period** ($t = 1, \ldots, N$): projected FCFFs discounted individually.
2. **Stable perpetuity** ($t > N$): terminal value (TV) computed via the Gordon Growth Model.

$$\text{EV} = \sum_{t=1}^{N} \frac{\text{FCFF}_t}{(1 + \text{WACC})^t} + \frac{\text{TV}_N}{(1 + \text{WACC})^N} \tag{6.1}$$

$$\text{TV}_N = \frac{\text{FCFF}_N \cdot (1 + g)}{(\text{WACC} - g)} \tag{6.2}$$

$$\text{Equity} = \text{EV} - \text{Net Debt} \tag{6.3}$$

$$P_{\text{share}} = \frac{\text{Equity}}{n_{\text{shares}}} \tag{6.4}$$

6.1.2 Implementation: double for Discounting, Rational for TV

Warning

The discount factor $(1+r)^{-t}$ is irrational for $t > 1$ in most cases. ratvalue uses `double` *only* for exponentiation; all other operations are exact rationals. In particular, the TV multiplier is an exact `Rational`:

$$\text{TV}_{\text{mul}} = \frac{(1 + g)/g_{\text{den}}}{(\text{WACC} - g)} = \frac{(g_{\text{den}} + g_{\text{num}}) \cdot \text{WACC}_{\text{den}}}{\text{WACC}_{\text{num}} \cdot g_{\text{den}} - g_{\text{num}} \cdot \text{WACC}_{\text{den}}}$$

6.1.3 API

```
struct DCFInputs {
    std::vector<Currency> projected_fcff;
    Rational              wacc;
    Rational              terminal_growth;
    Currency              net_debt;
    int64_t               shares_outstanding;
};

struct DCFResult {
    Currency enterprise_value;
    Currency equity_value;
    Currency equity_value_per_share;
    Currency pv_fcff;
    Currency terminal_value;      // TV not yet discounted (exact
       Rational)
    Currency pv_terminal_value;
};

expected<DCFResult, ValuationError>
compute_dcf(const DCFInputs& inputs);
```

6.2 DDM — Dividend Discount Model

The DDM replaces FCFF with dividends and discounts at K_e:

```
struct DDMInputs {
    Currency                    base_dividend_per_share;
    std::vector<ProjectionStage> stages;
    Rational                    cost_of_equity;
    Rational                    stable_growth;
};

expected<DDMResult, ValuationError>
compute_ddm(const DDMInputs& inputs);
```

6.3 H-Model (Fuller & Hsia, 1984)

The H-Model is a closed-form solution for growth that *declines linearly* from g_H to g_L over $2H$ years:

$$P = \frac{D_0 \cdot [(1 + g_L) + H \cdot (g_H - g_L)]}{K_e - g_L} \tag{6.5}$$

The entire calculation is *exact* in rational arithmetic — no temporal discounting is involved.

```
struct HModelInputs {
    Currency base_dividend;
    Rational high_growth;     // g_H
    Rational stable_growth;   // g_L
    Rational half_life;       // H in years (e.g., {5,1})
    Rational cost_of_equity;
};

```

```
struct HModelResult {
    Currency intrinsic_value;
    Rational tv_multiplier;   // for auditing
};

expected<HModelResult, ValuationError>
compute_h_model(const HModelInputs& inputs);
```

Chapter 7

Terminal Value

7.1 Consistency with ROIC

7.1.1 The Problem with the Standard Gordon Model

The Gordon Growth TV implies a reinvestment rate:

$$\mathrm{RR}_{\mathrm{impl}} = \frac{g}{\mathrm{ROIC}_{\mathrm{stable}}}$$

If the analyst assumes $g = 3\%$ but $\mathrm{ROIC}_{\mathrm{stable}} = 8\%$, then $\mathrm{RR}_{\mathrm{impl}} = 37.5\%$. If the projected FCFFs used RR $= 30\%$, there is a discontinuity at the terminal stage that inflates the TV.

7.1.2 Consistent Terminal Value (Damodaran ch. 12)

$$\mathrm{TV} = \frac{\mathrm{NOPAT}_{\mathrm{stable}} \cdot (1 - g/\mathrm{ROIC}_{\mathrm{stable}})}{\mathrm{WACC} - g} \tag{7.1}$$

Uses the terminal-period NOPAT as the base, deriving the stable FCFF from the reinvestment rate consistent with the assumed ROIC.

7.1.3 Consistency Audit

```
struct TerminalConsistency {
    Rational implied_reinvestment_rate;  // g / ROIC_stable
    Rational return_spread;              // ROIC_stable - WACC
    bool     value_creating;             // ROIC > WACC
    bool     reinvestment_feasible;      // 0 <= g/ROIC <= 1
};

TerminalConsistency check_terminal_consistency(
    Rational wacc, Rational terminal_growth, Rational terminal_roic);

struct ConsistentTVInputs {
    Currency stable_nopat;
    Rational wacc;
    Rational terminal_growth;
    Rational terminal_roic;
};

```

```
expected<Currency, ValuationError>
consistent_terminal_value(const ConsistentTVInputs& inputs);
```

Warning

When $\text{ROIC}_{\text{stable}} < \text{WACC}$, the TV is still positive but the firm destroys value in the perpetuity (`value_creating = false`). This is economically valid; the model does not reject this case.

7.2 TV by Multiple

An alternative to the Gordon Growth model, anchoring the TV to an observed market multiple:

$$\text{TV} = \text{EBITDA}_N \times \text{EV/EBITDA}_{\text{sector}} \tag{7.2}$$

```
expected<Currency, ValuationError>
terminal_value_by_multiple(Currency terminal_ebitda,
                           Rational ev_ebitda_multiple);
```

Chapter 8

APV — Adjusted Present Value

8.1 Motivation

The APV explicitly separates the unlevered firm value from the tax shield benefit:

$$\text{APV} = V_U + PV(\text{Tax Shields}) \tag{8.1}$$

Where V_U is the unlevered firm value, obtained by discounting FCFFs at the unlevered cost of equity K_u:

$$K_u = R_f + \beta_U \cdot \text{ERP} + \lambda \cdot \text{CRP}$$

APV is preferred over WACC when the capital structure changes over time (LBOs, recapitalizations, projects with amortization schedules).

8.2 Modigliani-Miller (1963): Permanent Debt

For permanent debt and risk-free tax shields (discounted at R_f):

$$PV(\text{TS})_{\text{MM}} = t \cdot D \tag{8.2}$$

```cpp
struct APVInputs {
    std::vector<Currency> projected_fcff;
    Rational              terminal_growth;
    Rational              unlevered_cost_of_equity;  // Ku
    Currency              debt_market_value;          // D
    Rational              tax_rate;                   // t
    Currency              net_debt;
    int64_t               shares_outstanding;
};

expected<APVResult, ValuationError>
compute_apv(const APVInputs& inputs);
```

8.3 Miles-Ezzell (1980): Rebalanced Debt

When the firm maintains a constant D/V ratio (rebalanced each period), tax shields bear the risk of assets and must be discounted at K_u, except the first period (known with certainty):

$$PV(\text{TS})_{\text{ME}} = \frac{t \cdot K_d \cdot D \cdot (1 + K_u)}{(1 + K_d)(K_u - g)} \tag{8.3}$$

The **Miles-Ezzell multiplier** is computed in exact rational arithmetic via `detail::r_mul` and `detail::r_div`.

```
struct APVMilesEzzellInputs {
    // ... all fields from APVInputs, plus:
    Rational cost_of_debt;   // required for ME formula
};

expected<APVResult, ValuationError>
compute_apv_miles_ezzell(const APVMilesEzzellInputs& inputs);
```

Example: MM vs. ME comparison (Petrobras)

$D = \text{R\$250B}$, $t = 34\%$, $K_d = 5.78\%$, $K_u \approx 11.57\%$, $g = 1.5\%$:

$$PV(\text{TS})_{\text{MM}} = 0.34 \times 250 = \text{R\$85B}$$

$$PV(\text{TS})_{\text{ME}} = \frac{0.34 \times 0.0578 \times 250 \times 1.1157}{1.0578 \times 0.1007} \approx \text{R\$51B}$$

MM overstates the tax shield benefit by $\approx 40\%$ in this case.

Chapter 9

EVA and Excess Returns

9.1 Theoretical Background

The excess returns model (Damodaran, ch. 13) decomposes firm value into:

$$V = \text{IC} + PV(\text{EVA}) \tag{9.1}$$

Where $\text{EVA}_t = (\text{ROIC}_t - \text{WACC}) \times \text{IC}_t$ and IC_t grows with reinvestment. For the single-stage (Gordon) stable case:

$$V = \text{IC} \cdot \frac{\text{ROIC} - g}{\text{WACC} - g} \tag{9.2}$$

DCF equivalence: when inputs are consistent ($g = \text{ROIC} \times \text{RR}$ and $\text{FCFF} = \text{NOPAT}(1 - \text{RR})$), equations (6.1) and (9.1) yield the same V. The EVA decomposition reveals *value of assets in place* vs. *value of future growth*.

9.2 Economic Interpretation

- ROIC > WACC: the firm creates value above its cost of capital; $PV(\text{EVA}) > 0$.
- ROIC = WACC: *neutral* growth; $V = \text{IC}$.
- ROIC < WACC: every dollar invested destroys value; $PV(\text{EVA}) < 0$.

9.3 API

```
struct SingleStageERInputs {
    Currency invested_capital;
    Rational roic;
    Rational wacc;
    Rational terminal_growth;
};

struct ExcessReturnsResult {
    Currency firm_value;
    Currency asset_value;        // = IC
    Currency pv_excess_returns;  // PV(EVA)
};
```

```
13
14 expected<ExcessReturnsResult, ValuationError>
15 excess_returns_value(const SingleStageERInputs& inputs);
16
17 struct MultiStageERInputs {
18     Currency                    base_invested_capital;
19     Rational                    roic;
20     std::vector<ProjectionStage> stages;
21     Rational                    wacc;
22     Rational                    terminal_roic;
23     Rational                    terminal_growth;
24 };
25
26 expected<ExcessReturnsResult, ValuationError>
27 excess_returns_multistage(const MultiStageERInputs& inputs);
```

Chapter 10

Justified Multiples

10.1 Background

Market multiples (P/E, P/BV, EV/EBITDA) are frequently interpreted without reference to fundamentals. Damodaran (ch. 17–20) derives the *justified multiple*: the value that would be correct given the firm's growth, profitability, and cost of capital.

$$P/E = \frac{\text{payout}}{K_e - g_{\text{equity}}} \tag{10.1}$$

$$P/BV = \text{ROE} \times \frac{\text{payout}}{K_e - g_{\text{equity}}} \tag{10.2}$$

$$EV/EBITDA = \frac{(1-t)(1-\text{RR})}{\text{WACC} - g} \tag{10.3}$$

Where $g_{\text{equity}} = \text{ROE} \times (1 - \text{payout})$ and $\text{RR} = g/\text{ROIC}$ (firm reinvestment rate).

10.2 API

```
struct JustifiedMultiplesInputs {
    Rational cost_of_equity;
    Rational wacc;
    Rational roe;
    Rational payout_ratio;
    Rational roic;
    Rational tax_rate;
};

struct JustifiedMultiples {
    Rational pe_ratio;
    Rational pb_ratio;
    Rational ev_ebitda;
    Rational g_equity;
    Rational implied_reinvestment;
};

expected<JustifiedMultiples, ValuationError>
compute_justified_multiples(const JustifiedMultiplesInputs& inputs);
```

Example: Petrobras — justified multiples (normalized ROE = 15%)

With $K_e = 11.57\%$, WACC $= 8.99\%$, $\text{ROE}_{\text{norm}} = 15\%$, payout $= 60\%$, ROIC $= 16.63\%$, $t = 34\%$:

$$g = 15\% \times 40\% = 6\%$$

$$P/E = \frac{0.60}{0.1157 - 0.06} = 10.8\times \qquad P/BV = 0.15\times 10.8 = 1.6\times \qquad EV/EBITDA = 14.1\times$$

Chapter 11

Optimal Capital Structure

11.1 Algorithm

The minimum cost of capital maximizes firm value. ratvalue sweeps debt ratios $d \in \{0/N, 1/N, \ldots, (N-1)/N\}$ and for each point:

1. Computes $D/E = d/(1-d)$ as `Rational`;
2. Levers $\beta_U \to \beta_L$ via Hamada (eq. 4.2);
3. Computes K_e via CAPM (eq. 4.1);
4. Iterates K_d from the ICR:
   (a) Interest $= D \times K_d$
   (b) ICR $=$ EBIT/Interest
   (c) $K_d \leftarrow R_f +$ spread(ICR)
   (d) Repeat until convergence ($< 10^{-9}$, typically 2–5 iterations)
5. Computes exact WACC as `Rational`.

The optimum is the point of minimum WACC.

11.2 API

```
struct OptimalCapitalStructureInputs {
    Rational  unlevered_beta;
    Rational  risk_free_rate;
    Rational  equity_risk_premium;
    Rational  country_risk_premium{0, 1};
    Rational  lambda{1, 1};
    Rational  tax_rate;
    Currency  ebit;        // for ICR computation
    Currency  firm_value; // E+D (base for D = d * FV)
    int       steps{10};
    bool      large_firm{true};
};

struct CapitalStructurePoint {
```

```
    Rational      debt_ratio;
    Rational      levered_beta;
    Rational      cost_of_equity;
    Rational      cost_of_debt;
    CreditRating  rating;
    double        interest_coverage;
    Rational      wacc;
};

struct OptimalCapitalStructureResult {
    std::vector<CapitalStructurePoint> schedule;
    CapitalStructurePoint              optimal;
};

expected<OptimalCapitalStructureResult, ValuationError>
optimal_capital_structure(const OptimalCapitalStructureInputs& inputs
    );
```

Warning

The Damodaran ICR model captures only the *default cost via spread*. Indirect costs of financial distress (customer loss, suboptimal hiring, management distraction) are not modeled. For firms with high EBIT relative to firm value, the algorithm may indicate high leverage ratios that would be impractical in reality.

Chapter 12

Specialized Models

12.1 Scenario-Based Valuation: Startups and Transitioning Firms

12.1.1 Motivation

Young firms or those undergoing technological transitions have strongly asymmetric outcome distributions. A single "base case" DCF ignores this dispersion. Damodaran (*Dark Side*, ch. 10–12) recommends:

1. Define n scenarios (typically 3–5) with probabilities;
2. Compute the TV of each scenario: $\text{TV}_i = \text{FCFF}_i \cdot \text{mul}_i$;
3. Discount each TV to the present: $PV_i = \text{TV}_i / (1 + \text{WACC}_i)^T$;
4. Apply a survival probability:

$$\text{EV}_{\text{adjusted}} = \left[\sum_i P_i \cdot PV_i \right] \cdot P(\text{survive}) + V_{\text{failure}} \cdot [1 - P(\text{survive})] \tag{12.1}$$

12.1.2 API

```
struct ValuationScenario {
    std::string name;
    Rational    probability;
    Currency    terminal_fcff;
    Rational    terminal_growth;
    Rational    wacc;
    int         years_to_terminal;
};

struct StartupValuationInputs {
    std::vector<ValuationScenario> scenarios;
    Rational  survival_probability;
    Currency  failure_value;
    Currency  net_debt;
    int64_t   shares_outstanding;
};
```

```
17
18 expected<StartupValuationResult, ValuationError>
19 startup_valuation(const StartupValuationInputs& inputs);
```

Probabilities must sum to 1.0 (tolerance 10^{-6}); otherwise `InvalidInput` is returned.

12.2 Equity as an Option: Merton Model

12.2.1 Background

In firms with high debt, equity behaves as a *call option* on the firm's assets, with a strike equal to the face value of debt (Merton, 1974):

$$d_1 = \frac{\ln(V/D) + (r + \sigma_V^2/2)T}{\sigma_V\sqrt{T}} \tag{12.2}$$

$$d_2 = d_1 - \sigma_V\sqrt{T} \tag{12.3}$$

$$E = V \cdot \mathcal{N}(d_1) - D \cdot e^{-rT} \cdot \mathcal{N}(d_2) \tag{12.4}$$

The *risk-neutral* probability of default is $\mathcal{N}(-d_2)$. The distance to default d_2 is the primary indicator of financial health.

Warning

V and σ_V are firm asset parameters, *not directly observable*. The iterative estimation (solving $\{E, \sigma_E\}$ from the nonlinear system $E = \text{BS}(V, \sigma_V)$ and $\sigma_E E = \mathcal{N}(d_1)\sigma_V V$) is not implemented; the user provides V and σ_V directly.

12.2.2 API

```
1  struct EquityAsOptionInputs {
2      double firm_value;         // V
3      double debt_face_value;    // D
4      double asset_volatility;   // sigma_V
5      double risk_free_rate;     // r (continuous)
6      double debt_maturity;      // T in years
7  };
8
9  struct EquityAsOptionResult {
10     double equity_value;
11     double debt_market_value;
12     double probability_default;  // N(-d2)
13     double distance_to_default;  // d2
14     double d1;
15 };
16
17 EquityAsOptionResult                          // no expected: always
       defined
18 equity_as_option(const EquityAsOptionInputs& inputs) noexcept;
```

The normal CDF is implemented via: $\mathcal{N}(x) = \frac{1}{2}\,\text{erfc}\left(-x/\sqrt{2}\right)$ (`std::erfc` from `<cmath>`).

12.3 FCFE for Financial Institutions

12.3.1 Why Banks Are Different

- Debt (deposits, certificates) is *raw material*, not a source of capex financing — computing FCFF or WACC makes no economic sense.
- Reinvestment is regulated: Basel III requires minimum capital against risk-weighted assets (RWA).
- The only relevant cost of capital is K_e.

12.3.2 Model

$$\text{FCFE}_{\text{bank}} = NI \times (1 - \text{RR}_{\text{equity}}) \tag{12.5}$$

$$\text{RR}_{\text{equity}} = \frac{\Delta\text{RWA} \times \text{target Tier1}}{NI}$$

12.3.3 API

```
struct BankFCFEInputs {
    Currency net_income;
    Rational equity_reinvestment_rate;
};

expected<Currency, ValuationError>
compute_bank_fcfe(const BankFCFEInputs& inputs);

struct RegulatoryCapitalInputs {
    Currency net_income;
    Currency current_rwa;
    Currency projected_rwa;
    Rational target_tier1_ratio;
};

expected<Rational, ValuationError>
bank_equity_reinvestment_rate(const RegulatoryCapitalInputs& inputs);
```

For a bank's DCF, use `compute_fcfe_dcf` with FCFEs generated by `compute_bank_fcfe`.

Chapter 13

Full Example: Petrobras and Itaú Unibanco

13.1 Data

Table 13.1: Petrobras S.A. — approximate 2023 data

Item	BRL billions	Code
Net revenue	500	`B(500)`
Historical EBIT 2019–2023	80, 95, 145, 165, 145	`B(80)...`
Normalized EBIT (average)	126	—
D&A	45	`B(45)`
CapEx	70	`B(70)`
ΔNWC	5	`B(5)`
Net Income	124	`B(124)`
Interest Expense	22	`B(22)`
Net Debt	250	`B(250)`
Invested Capital	500	`B(500)`
Shares outstanding	13 billion	`13e9`

13.2 Cost of Capital Parameters

- $R_f = 5\%$ (US 10Y T-Bond, *stripped*);
- ERP = 5% (US market, Damodaran 2024);
- CRP = 3% (Brazil, EMBI+ adjusted for volatility);
- $\lambda = 0.5$ (Petrobras: $\approx 50\%$ of revenues from exports);
- $\beta_U = 0.8$ (O&G sector, bottom-up from Shell, ExxonMobil, BP, Chevron, Equinor with $t_{\text{sector}} = 25\%$);
- $D/E = 250/500 = 0.5$; $t = 34\%$.

Cost of capital results: $\beta_L \approx 1.014$; $K_e \approx 11.57\%$; ICR= $145/22 = 6.6\times \rightarrow$ AA rating $\rightarrow K_d = 5.78\%$; WACC $\approx 8.99\%$ (exact rational: 44933/500000).

13.3 Growth and FCFF

$$\begin{aligned} \text{FCFF}_0 &= 126 \times 0.66 + 45 - 70 - 5 = \text{BRL}\,53.2\text{B} \\ \text{ROIC} &= 83.2/500 = 16.63\% \\ \text{RR} &= 30/83.2 = 36.08\% \\ g_{\text{fund}} &= 16.63\% \times 36.08\% = 6\% \end{aligned}$$

The DCF uses $g_{\text{explicit}} = 3\%$ for 5 years and $g_{\text{terminal}} = 1.5\%$.

13.4 Summary of Results

Table 13.2: Model comparison — Petrobras (PETR4)

Model	Fair value	Module
DCF (FCFF, Gordon TV)	R$39.88	`dcf`
FCFE (direct equity)	R$85.72	`fcfe`
APV Modigliani-Miller	R$31.13	`apv`
APV Miles-Ezzell	R$28.55	`apv`
H-Model (dividends)	R$54.11	`ddm`
Scenarios (energy trans.)	R$24.42	`startup`
EVA / Excess Returns	R$58.51	`excess_returns`
Market price (2024)	R$38–42	—

Warning

The divergence across models is *expected and informative*:

- **FCFE** uses 2023 net income (R$124B) without normalization — inflated by the high oil price cycle;
- **APV** does not embed the debt benefit into the growth rates, yielding a lower TV;
- The **scenario** model heavily penalizes accelerated decarbonization (20% probability of FCFF falling to R$40B);
- **DCF/FCFF normalized** (R$39.88) is the most consistent with the market price because it uses the mid-cycle average EBIT.

13.5 Itaú Unibanco (ITUB4)

The high $\text{RR}_{\text{equity}}$ reflects rapid growth in Brazilian credit: the bank must retain $\approx 56\%$ of earnings to support the regulatory expansion of its asset base.

Table 13.3: Itaú Unibanco — approximate 2023 data

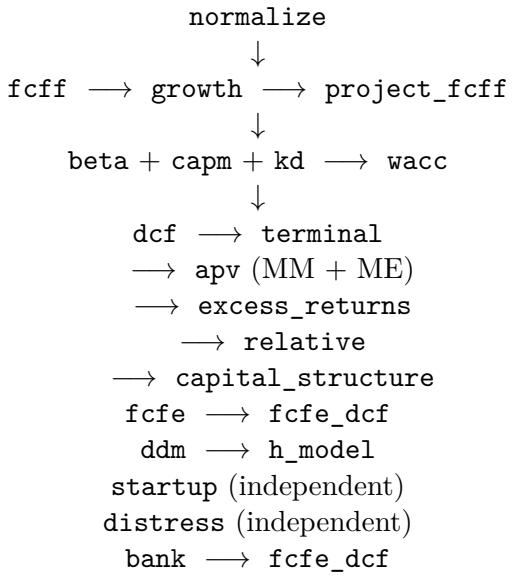
Item	**Value**
Net Income	R$35B
Current RWA	R$1,800B
Projected RWA	R$1,944B (+8%)
Target Tier1	13.5%
ΔRegulatory capital	R$144B $\times$ 13.5% = R$19.4B
RR_{equity}	R$19.4B / R$35B = 55.5%
$FCFE_{bank}$	R$35B $\times$ 44.5% = R$15.6B
Fair value (DCF)	R$22.61/share

Chapter 14

Data Flow Between Modules

14.1 Overview

The diagram below illustrates the typical flow of a complete analysis:



14.2 Cross-Model Consistency

For FCFF-DCF and FCFE-DCF to yield the same share price:

1. FCFE = FCFF − Interest$(1-t)$ + ΔD;
2. Both must use the same growth rate assumption;
3. ΔD must be consistent with the D/V ratio over time.

For APV and WACC-DCF to yield the same EV:

- WACC assumes constant leverage (adjusts WACC each period);
- APV assumes a predetermined debt schedule.
- In general, APV is more flexible; WACC is simpler for stable structures.

Chapter 15

Python Bindings

15.1 Overview

The `python/` directory provides nanobind bindings that expose the complete ratvalue API to Python without rewriting any logic. All 18 modules are covered. The C++ library performs every calculation; Python is only the calling layer.

Warning

The Python boundary converts `ratmoney::Currency` and `ratmoney::Rational` to and from `double`. This introduces floating-point rounding at the interface, but all internal arithmetic remains exact. For production-grade auditing use the C++ API directly.

15.2 Installation

Prerequisites:

- Python $\geq$ 3.9
- nanobind $\geq$ 2.0 and scikit-build-core $\geq$ 0.9
- The C++ library and ratmoney already installed in `/usr/local`

Option 1a — pip inside a virtual environment (recommended):

```
python3 -m venv venv && source venv/bin/activate
pip install nanobind scikit-build-core
pip install --no-build-isolation .
```

Option 1b — pip system-wide on Arch Linux / PEP 668 systems:

```
pip install --break-system-packages nanobind scikit-build-core
pip install --break-system-packages --no-build-isolation .
```

The `-no-build-isolation` flag is required so that pip uses the already-installed `nanobind` and `scikit-build-core` instead of attempting to fetch them into an isolated subprocess environment. The `-break-system-packages` flag is additionally required on systems enforcing PEP 668 (e.g. Arch Linux) when installing outside a virtual environment.

Option 2 — manual CMake build:

```
cmake -S . -B build_py \
  -DCMAKE_BUILD_TYPE=Release \
  -DBUILD_PYTHON=ON \
  -DCMAKE_PREFIX_PATH="/usr/local"
cmake --build build_py -j$(nproc)
# shared library in build_py/python/ratvalue.cpython-*.so
```

The nanobind cmake directory is located automatically via the active Python interpreter inside `python/CMakeLists.txt`; no manual path is needed.

15.3 Unit Conventions

Table 15.1: Python boundary unit conventions

Value type	Unit at Python boundary
Aggregate monetary (EV, equity, IC, debt, ...)	BRL billions (`float`)
Per-share values (dividends, intrinsic price)	BRL reais (`float`)
Rates (Ke, Kd, WACC, growth, tax, ...)	Decimal (`0.05` = 5%)
Beta, ratios, multiples	Plain `float`

Internally, `bind_helpers.h` converts via:

Listing 15.1: Python boundary converters (bind_helpers.h)

```
// BRL billions -> Currency (1e11 cents per billion)
inline Currency b2c(double billions) {
    int64_t units = static_cast<int64_t>(std::round(billions * 1e11))
        ;
    return {units, UNIT_RATE(), BRL_DESC()};
}

// float rate (e.g. 0.05) -> Rational with denominator 100,000
inline Rational d2r(double v, int64_t den = 100'000) {
    return {static_cast<int64_t>(std::round(v * den)), den};
}
```

15.4 API Reference

15.4.1 Cost of Capital

```
ke = cost_of_equity(rf, beta, erp, crp=0.0, lam=1.0) -> float

result = synthetic_cost_of_debt(ebit_billions, interest_billions,
                                rf, large_firm=True) ->
                                   SyntheticKdResult
# .cost_of_debt  .default_spread  .rating (str)  .interest_coverage

wacc = compute_wacc(equity_weight, ke, debt_weight, kd, tax_rate) ->
   float

bl = lever_beta(unlevered_beta, debt_to_equity, tax_rate) -> float
```

```
bu = unlever_beta(levered_beta, debt_to_equity, tax_rate) -> float
b  = bottom_up_beta(comparable_levered_betas,
    comparable_debt_to_equity,
                    comparable_tax_rate,
                    target_debt_to_equity, target_tax_rate) -> float
```

15.4.2 Data and Cash Flows

```
ebit = normalize_ebit_avg(historical_ebit_billions)          -> float
ebit = normalize_ebit_margin(current_revenue_billions,
                             normalized_margin)               -> float

fcff  = compute_fcff(ebit_b, tax_rate, da_b, capex_b,
                     delta_nwc_b=0.0)                        -> float
fcffs = project_fcff(base_fcff_billions,
                     stages: list[tuple[float,int]])         -> list[
                        float]

fcfe  = compute_fcfe(net_income_b, debt_ratio, capex_b,
                     da_b, delta_nwc_b=0.0,
                     net_new_debt_b=0.0)                     -> float
result = compute_fcfe_dcf(projected_fcfe_billions,
                          cost_of_equity, terminal_growth,
                          shares)                            ->
                             FCFEDCFResult
```

15.4.3 Growth

```
roic = compute_roic(nopat_billions, invested_capital_billions) ->
    float
roe  = compute_roe(net_income_billions, book_equity_billions)  ->
    float
rr   = compute_reinvestment_rate(capex_b, da_b,
                                 delta_nwc_b, nopat_b)        ->
                                    float
g    = fundamental_growth_firm(roic, reinvestment_rate)        ->
    float
g    = fundamental_growth_equity(roe, retention_ratio)         ->
    float
```

15.4.4 Valuation Models

```
result = compute_dcf(projected_fcff_billions, wacc,
                     terminal_growth, net_debt_billions,
                     shares)                                 ->
                        DCFResult
# .enterprise_value  .equity_value  .price_per_share
# .pv_fcff  .pv_terminal_value

result = compute_ddm(base_dividend, stages, cost_of_equity,
                     stable_growth)                          ->
                        DDMResult
result = compute_h_model(base_dividend, high_growth,
```

```
                            stable_growth, half_life,
                            cost_of_equity)                    ->
                               HModelResult

tc = check_terminal_consistency(wacc, terminal_growth,
                                terminal_roic)                 ->
                                   TerminalConsistency
tv = consistent_terminal_value(stable_nopat_billions, wacc,
                               terminal_growth,
                               terminal_roic)                  -> float
tv = terminal_value_by_multiple(terminal_ebitda_billions,
                                ev_ebitda_multiple)            -> float

result = compute_apv(projected_fcff_billions, terminal_growth,
                     ku, debt_billions, tax_rate,
                     net_debt_billions, shares)                ->
                        APVResult
result = compute_apv_miles_ezzell(projected_fcff_billions,
                                  terminal_growth, ku, kd,
                                  debt_billions, tax_rate,
                                  net_debt_billions,
                                  shares)                      ->
                                     APVResult
# .unlevered_firm_value  .pv_tax_shield  .apv
# .equity_value  .price_per_share

result = excess_returns_value(invested_capital_billions,
                              roic, wacc,
                              terminal_growth)                 ->
                                 ExcessReturnsResult
result = excess_returns_multistage(invested_capital_billions,
                                   roic, stages, wacc,
                                   terminal_roic,
                                   terminal_growth)            ->
                                      ExcessReturnsResult

result = compute_justified_multiples(ke, wacc, roe,
                                     payout_ratio, roic,
                                     tax_rate)                 ->
                                        JustifiedMultiples
# .pe_ratio  .pb_ratio  .ev_ebitda  .g_equity  .implied_reinvestment

result = optimal_capital_structure(
             unlevered_beta, rf, erp, tax_rate,
             ebit_billions, firm_value_billions,
             crp=0.0, lam=1.0, steps=10,
             large_firm=True)                  ->
                OptimalCapitalStructureResult
# .schedule: list[CapitalStructurePoint]
# .optimal:  CapitalStructurePoint
#   each point: .debt_ratio .cost_of_equity .cost_of_debt
#               .rating (str) .interest_coverage .wacc

result = startup_valuation(scenarios, survival_probability,
                           failure_value_billions,
                           net_debt_billions, shares)          ->
                              StartupValuationResult
# scenarios: list[dict] with keys:
```

```
#   name, probability, terminal_fcff_b, terminal_growth,
#   wacc, years_to_terminal

result = equity_as_option(firm_value, debt_face_value,
                          asset_volatility, risk_free_rate,
                          debt_maturity_years)             ->
                             EquityAsOptionResult
# .equity_value  .debt_market_value
# .probability_default  .distance_to_default  .d1

rr     = bank_equity_reinvestment_rate(net_income_b,
             current_rwa_b, projected_rwa_b,
             target_tier1_ratio)                           -> float
fcfe_b = compute_bank_fcfe(net_income_billions,
                           equity_reinvestment_rate)       -> float
```

15.5 Complete Example: Petrobras

Listing 15.2: Full DCF in Python — results match C++ binary

```
import ratvalue as rv

# Cost of capital
ke    = rv.cost_of_equity(rf=0.05, beta=1.014, erp=0.05, crp=0.03, lam
    =0.5)
kd_r = rv.synthetic_cost_of_debt(ebit_billions=145, interest_billions
    =22, rf=0.05)
wacc = rv.compute_wacc(equity_weight=2/3, ke=ke,
                       debt_weight=1/3,   kd=kd_r.cost_of_debt,
                          tax_rate=0.34)
# ke=11.57%  kd=5.78% (AA, ICR=6.6x)  wacc=8.99%

# FCFF
fcff0 = rv.compute_fcff(ebit_b=126, tax_rate=0.34, da_b=45,
                        capex_b=70, delta_nwc_b=5)    # 53.16B
fcffs = rv.project_fcff(fcff0, [(0.03, 5)])

# Fundamental growth
roic = rv.compute_roic(nopat_billions=83.2, invested_capital_billions
    =500)
rr   = rv.compute_reinvestment_rate(capex_b=70, da_b=45,
                                    delta_nwc_b=5, nopat_b=83.2)
g    = rv.fundamental_growth_firm(roic, rr)            # 6.0%

# Terminal value consistency check
tc = rv.check_terminal_consistency(wacc=wacc, terminal_growth=0.015,
                                   terminal_roic=0.12)
assert tc.value_creating and tc.reinvestment_feasible

# DCF
dcf = rv.compute_dcf(fcffs, wacc=wacc, terminal_growth=0.015,
                     net_debt_billions=250, shares=13_000_000_000)
# dcf.price_per_share -> 39.89  (market: R$38-42)

# APV comparison
apv_mm = rv.compute_apv(fcffs, 0.015, ku=ke, debt_billions=250,
```

```
                            tax_rate=0.34, net_debt_billions=250,
                            shares=13_000_000_000)
apv_me = rv.compute_apv_miles_ezzell(fcffs, 0.015, ku=ke,
                                     kd=kd_r.cost_of_debt,
                                     debt_billions=250, tax_rate
                                        =0.34,
                                     net_debt_billions=250,
                                     shares=13_000_000_000)
# apv_mm.pv_tax_shield -> 85.0B (MM)
# apv_me.pv_tax_shield -> 51.4B (ME, more conservative)

# H-Model
hm = rv.compute_h_model(base_dividend=5.0, high_growth=0.03,
                            stable_growth=0.015, half_life=5,
                            cost_of_equity=ke)
# hm.intrinsic_value -> 54.12

# EVA
er = rv.excess_returns_value(invested_capital_billions=500,
                                roic=roic, wacc=wacc, terminal_growth
                                   =0.015)
# er.firm_value -> 1010.6B   er.pv_excess_returns -> 510.6B

# Justified multiples (normalized ROE=15%)
jm = rv.compute_justified_multiples(ke=ke, wacc=wacc, roe=0.15,
                                    payout_ratio=0.60, roic=roic,
                                    tax_rate=0.34)
# jm.pe_ratio=10.8x  jm.pb_ratio=1.6x  jm.ev_ebitda=14.1x

# Scenario valuation (energy transition risk)
scenarios = [
    {"name":"Bull","probability":0.35,"terminal_fcff_b":90,
     "terminal_growth":0.02,"wacc":0.085,"years_to_terminal":5},
    {"name":"Base","probability":0.45,"terminal_fcff_b":60,
     "terminal_growth":0.015,"wacc":0.090,"years_to_terminal":5},
    {"name":"Bear","probability":0.20,"terminal_fcff_b":30,
     "terminal_growth":0.010,"wacc":0.100,"years_to_terminal":5},
]
sc = rv.startup_valuation(scenarios, survival_probability=0.95,
                          failure_value_billions=100,
                          net_debt_billions=250, shares=13
                             _000_000_000)
# sc.price_per_share -> 25.59

# Merton distress model (2015 crisis scenario)
dist = rv.equity_as_option(firm_value=350, debt_face_value=420,
                           asset_volatility=0.45, risk_free_rate
                              =0.12,
                           debt_maturity_years=3)
# dist.equity_value=128.4B   dist.probability_default=56.4%
```

Appendix A

Rating and Spread Tables (Damodaran 2023)

Left: *large cap.* Right: *small cap.*

Table A.1: Large Cap and Small Cap

Rating	Min. ICR	Spread (bps)
Aaa/AAA	8.50	63
Aa2/AA	6.50	78
A1/A+	5.50	98
A2/A	4.25	108
A3/A–	3.00	122
Baa2/BBB	2.50	156
Ba1/BB+	2.25	200
Ba2/BB	2.00	240
B1/B+	1.75	350
B2/B	1.50	375
B3/B–	1.25	500
Caa/CCC	0.80	600
Ca2/CC	0.65	700
C2/C	0.20	800
D2/D	< 0.20	1200

Rating	Min. ICR	Spread (bps)
Aaa/AAA	12.50	63
Aa2/AA	9.50	78
A1/A+	7.50	98
A2/A	6.00	108
A3/A–	4.50	122
Baa2/BBB	4.00	156
Ba1/BB+	3.50	200
Ba2/BB	3.00	240
B1/B+	2.50	350
B2/B	2.00	375
B3/B–	1.50	500
Caa/CCC	1.25	600
Ca2/CC	0.80	700
C2/C	0.50	800
D2/D	< 0.50	1200

Appendix B

References and Further Reading

Bibliography

[1] Damodaran, A. (2012). *Investment Valuation: Tools and Techniques for Determining the Value of Any Asset*. 3rd ed. Wiley Finance.

[2] Damodaran, A. (2009). *The Dark Side of Valuation*. 2nd ed. FT Press. [3rd ed., 2018]

[3] Miles, J. A.; Ezzell, J. R. (1980). The Weighted Average Cost of Capital, Perfect Capital Markets, and Project Life: A Clarification. *Journal of Financial and Quantitative Analysis*, 15(3), 719–730.

[4] Modigliani, F.; Miller, M. H. (1963). Corporate Income Taxes and the Cost of Capital: A Correction. *American Economic Review*, 53(3), 433–443.

[5] Merton, R. C. (1974). On the Pricing of Corporate Debt: The Risk Structure of Interest Rates. *Journal of Finance*, 29(2), 449–470.

[6] Fuller, R. J.; Hsia, C. C. (1984). A Simplified Common Stock Valuation Model. *Financial Analysts Journal*, 40(5), 49–56.

[7] Hamada, R. S. (1972). The Effect of the Firm's Capital Structure on the Systematic Risk of Common Stocks. *Journal of Finance*, 27(2), 435–452.

[8] Damodaran, A. *Damodaran Online* — data and tables updated annually. https://pages.stern.nyu.edu/~adamodar/

[9] *ratmoney* — C++23 library for exact rational monetary arithmetic. https://github.com/sheep-farm/ratmoney

[10] *ratvalue* is licensed under the MIT License. https://opensource.org/licenses/MIT